果壳 Cache 验证报告

Open Verify 学习任务

2026-09-08

1 验证结论

本次验证使用 Picker 生成 Python DUT，使用 Toffee Agent、参考模型和 pytest 对 NutShell Cache 的 SimpleBus 接口进行了验证。最终 `make report` 结果为 12 个测试全部通过；Verilator RTL 行覆盖率为 80.1%（1164/1454）。测试报告位于 `reports/cache`，RTL 覆盖率位于 `reports/rtl`，波形为 `cache.fst`。

在复位、读缺失填充、读命中、写回读、字节掩码、MMIO 路由、连续字访问和可复现混合事务场景下，DUT 与参考模型行为一致。

2 验证对象和环境

验证对象为 `rtl/Cache.v` 的 Cache 模块。Cache 通过 SimpleBus 接收上游请求，并将普通地址路由到 backing memory，将 MMIO 地址路由到 MMIO 从设备。环境由以下部分组成：

- Picker 生成的 Python DUT 和时钟/复位驱动；
- Toffee SimpleBus Master Agent；
- 普通内存与 MMIO 两个 SimpleBus responder；
- Cache 参考模型和 pytest 测试用例；
- responder 的事务记录，用于检查下游端口选择和 refill 行为。

验证边界包括 64 bit word、8 byte 写掩码、4 路组相联 Cache、按 line refill、普通内存访问和 MMIO 隔离。

3 测试点和用例

用例	测试点与通过判据
`reset`	复位后 `io_empty=1`，且没有下游事务。
`miss-hit`	首次读访问 backing memory，重复读返回相同数据且不增加 memory read 次数。
`write-mask`	全字写和单 byte 掩码写后读回数据正确。
`mmio-route`	`0x30000000` 写后读回 `0xCAFE`，普通 memory 计数不变。

用例	测试点与通过判据
`line-order`	一条 line 内 8 个相邻 word 逐一写入并逐一读回。
`mixed`	固定种子 `0xCAFE` 产生 64 笔混合读写，逐笔与 shadow model 比较。
`dirty-eviction`	四路冲突替换触发 dirty line burst writeback。
`backpressure`	下游 ready/valid 延迟，并在 refill burst 中途暂停后恢复。
`probe`	coherence probe hit/miss 与整条 Cache line release。
`cross-line`	跨 64B Cache line 边界的读写保持独立。
`random`	固定种子 128 笔混合读写、byte mask 和冲突替换。
`smoke`	官方基础 smoke 场景通过。

4 参考模型修正

验证过程中修正了示例环境中的两个问题：

1. 写请求原来用位置参数构造 `ReqMsg`，导致写数据落入 size 字段；现改为显式传入 `mask` 和 `data`。
2. 写响应原来固定返回 0；Cache 写命中返回写入前的 word，参考模型现保存旧值后再更新 shadow data。

上述修改只涉及验证环境和参考模型，不改变 `rtl/Cache.v`。

5 结果与覆盖率

复现命令：

```
cd /mnt/e/workspace/chip/open_verif/workspace/nutshell_cache
source ../../.tooling/env.sh
make report
```

结果为：**12 passed**；RTL line coverage 为 **80.1% (1164/1454)**。未达到 100% 是预期的，本次任务未覆盖 coherence、victim buffer、完整冲突替换矩阵和全部内部状态组合。

6 限制和结论

当前 RTL 对 `io_flush` 会触发 “only allow to flush icache” 设计断言，因此没有把数据 Cache flush 作为通过用例，而将其记录为接口约束。多主设备 coherence、victim 输出和跨 line 复杂替换也留作后续扩展。

综上，当前代码已经形成可复现的 Cache 验证环境，功能回归和 RTL 覆盖率报告均可由 `make report` 重新生成，满足新手任务要求的验证代码、测试点、结果分析和结论材料。